


```
// File: Lab01_Basic_Graph_Operations.c

// Problem 2: Write a program to perform the following basic graph-related operations:
// 1. Represent a graph using adjacency matrix.
// 2. Represent a graph using adjacency list.
// 3. Find the degree of each vertex in the graph.
// 4. Determine whether the graph is simple, complete, or connected.
// 5. Display total number of vertices and edges.
// Input: V = 5, edges [][] = {(0, 1), (0,2), (1,2), (2,3), (3,4)}
// Output:
// 1. Adjacency Matrix
// 2. Adjacency List
// 3. Degree of each vertex
// 4. Graph properties
// 5. Total vertices and edges
#include <stdio.h>
#include <stdlib.h>

void main()
{
    int v, e;

    printf("Enter no. of vertices");
    scanf("%d", &v);

    printf("Enter no. of edges");
    scanf("%d", &e);

    int add[v][v];

    for (int i = 0; i < v; i++)
    {
        for (int j = 0; j < v; j++)
        {
            add[i][j]=0;
        }
    }
    int n1,n2;

    for (int i = 0; i < e; i++)
    {
        printf("Enter Edges");
        scanf("%d %d",&n1,&n2);
        add[n1][n2]=1;
        add[n2][n1]=1;
    }

    for (int i = 0; i < v; i++){
        printf("%d -> ", i);
        for (int j = 0; j < v; j++)
        {
            if(add[i][j]==1){
                printf("%d ",j);
            }
        }
        printf("\n");
    }
    printf("Adjacency Matrix \n");
    for (int i = 0; i < v; i++){
        for (int j = 0; j < v; j++)
        {
            printf("%d ",add[i][j]);
        }
        printf("\n");
    }
}

//0 1 0 0
//1 0 1 1
//0 1 0 1
//0 1 1 0
```

```

// File: Lab02_Breadth_First_Search.c

// Objective: To implement Breadth First Search traversal for a graph.
// Problem: Perform a Breadth First Search (BFS) traversal starting from vertex 0.
// Input: adj[][] = [[1,2], [0,2,3], [0,1,4], [1,4], [2,3]]
// Output:
// [0, 1, 2, 3, 4]
#include<stdio.h>
void main(){

    int adj[5][5]={
        {0,1,1,0,0},
        {1,0,1,1,0},
        {1,1,0,0,1},
        {0,1,0,0,1},
        {0,0,1,1,0}
    };
    int visited[5]={0};
    int queue[5];
    int f=0,r=0;

    visited[0]=1;
    queue[r++]=0;

    while(f<r){
        int v=queue[f++];

        printf("%d ",v);

        for(int i=0;i<5;i++){
            if(adj[v][i] && !visited[i]){
                visited[i]=1;
                queue[r++]=i;
            }
        }
    }
}

```

```

// File: Lab03_Depth_First_Search.c

// Objective: To implement Depth First Search traversal for a graph.
// Problem: Write a program to perform DFS traversal starting from a given source vertex.
// Input: adj[][] = [[1,2], [0,2], [0,1,3,4], [2], [2]]
// Output:
// [0, 1, 2, 3, 4]
#include <stdio.h>

#define V 5

int graph[V][V] = {
    {0,1,1,0,0},
    {1,0,1,0,0},
    {1,1,0,1,1},
    {0,0,1,0,0},
    {0,0,1,0,0}
};

int visited[V];

void DFS(int v){
    visited[v] = 1;
    printf("%d ", v);

    for(int i=0; i<V; i++){
        if(graph[v][i] && !visited[i]){
            DFS(i);
        }
    }
}

int main(){
    int source = 0;

    printf("DFS Traversal: ");
    DFS(source);

    return 0;
}

```



```
// File: Lab06_Articulation_Points.c

// Objective: To identify articulation points in an undirected graph and determine connectivity and separability of a graph.
// Problem: Write a program to find all cut-vertices of a graph.
// Input: Connected graph with vertices and edges.
// Output:
// List of articulation points.
#include <stdio.h>
#include <stdbool.h>

#define MAX 100
int graph[MAX][MAX];
int visited[MAX], disc[MAX], low[MAX], parent[MAX];
bool ap[MAX];
int V, E;
int time = 0;

void DFS(int u)
{
    int children = 0;
    visited[u] = 1;
    disc[u] = low[u] = ++time;

    for (int v = 0; v < V; v++)
    {
        if (graph[u][v])
        {
            if (!visited[v])
            {
                children++;
                parent[v] = u;
                DFS(v);

                if (low[v] < low[u])
                    low[u] = low[v];

                if (parent[u] == -1 && children > 1)
                    ap[u] = true;

                if (parent[u] != -1 && low[v] >= disc[u])
                    ap[u] = true;
            }
            else if (v != parent[u])
            {
                if (disc[v] < low[u])
                    low[u] = disc[v];
            }
        }
    }
}

int main()
{
    printf("Enter number of vertices: ");
    scanf("%d", &V);

    printf("Enter number of edges: ");
    scanf("%d", &E);

    for (int i = 0; i < V; i++)
    {
        visited[i] = 0;
        parent[i] = -1;
        ap[i] = false;

        for (int j = 0; j < V; j++)
            graph[i][j] = 0;
    }

    printf("Enter edges (u v):\n");
    for (int i = 0; i < E; i++)
    {
        int u, v;
        scanf("%d %d", &u, &v);

        graph[u][v] = 1;
        graph[v][u] = 1;
    }

    for (int i = 0; i < V; i++)
    {
        if (!visited[i])
            DFS(i);
    }

    printf("\nArticulation Vertices :\n");

    int found = 0;
    for (int i = 0; i < V; i++)
    {
        if (ap[i])
        {
            printf("%d ", i);
            found = 1;
        }
    }

    if (!found)
        printf("No Articulation Vertices");

    printf("\n");

    return 0;
}
```



```

// File: Lab08_Dominating_Set.c

// Objective: To determine the dominating set and domination number of a graph.
// Problem: Write a program to:
// 1. Find dominating sets
// 2. Find domination number
// 3. Find total dominating sets
// 4. Find total domination number
// Input: N = 4, M = 4, Edges = {(1,2), (1,3), (3,4), (2,4)}
// Output:
// Possible Dominating Sets:
// {1,4}
// {2,3}
// {1,2}
// {1,3}
// {2,4}
// {3,4}
// Domination Number = 2
// Total Dominating Sets: {1,3} {2,4}
// Total Domination Number = 2
#include <stdio.h>

#define N 4

int graph[N][N] = {
    {0,1,1,0},
    {1,0,0,1},
    {1,0,0,1},
    {0,1,1,0}
};

int countBits(int mask){
    int c=0;
    while(mask){
        c += mask & 1;
        mask >>=1;
    }
    return c;
}

int isDominating(int mask){
    for(int v=0; v<N; v++){
        if(mask & (1<<v)) continue;

        int covered=0;
        for(int u=0; u<N; u++){
            if((mask&(1<<u)) && graph[u][v]){
                covered=1;
                break;
            }
        }
        if(!covered) return 0;
    }
    return 1;
}

int isTotalDominating(int mask){
    for(int v=0; v<N; v++){
        int covered=0;
        for(int u=0; u<N; u++){
            if((mask&(1<<u)) && graph[u][v]){
                covered=1;
                break;
            }
        }
        if(!covered) return 0;
    }
    return 1;
}

void printSet(int mask){
    printf("{");
    int first=1;
    for(int i=0;i<N;i++){
        if(mask&(1<<i)){
            if(!first) printf(",");
            printf("%d",i+1);
            first=0;
        }
    }
    printf("}");
}

int main(){
    int minDom=N+1, minTotal=N+1;

    printf("Possible Dominating Sets:\n");
    for(int mask=1; mask<(1<<N); mask++){
        if(isDominating(mask)){
            printSet(mask);
            printf("\n");
            int sz=countBits(mask);
            if(sz<minDom) minDom=sz;
        }
    }

    printf("\nDomination Number = %d\n\n",minDom);

    printf("Total Dominating Sets:\n");
    for(int mask=1; mask<(1<<N); mask++){
        if(isTotalDominating(mask)){
            printSet(mask);
            printf("\n");
            int sz=countBits(mask);
            if(sz<minTotal) minTotal=sz;
        }
    }

    printf("\nTotal Domination Number = %d\n",minTotal);

    return 0;
}

```

```

// File: Lab09_Total_Dominating_Set.c

// Objective: To determine the total dominating set and total domination number of a graph.
// Problem: Write a program to:
// 1. Find total dominating sets
// 2. Find total domination number
// Input: N = 4, M = 4, Edges = {(1,2), (1,3), (3,4), (2,4)}
// Output:
// Total Dominating Sets: {1,3} {2,4}
// Total Domination Number = 2
#include <stdio.h>

#define N 4

int graph[N][N] = {
    {0,1,1,0},
    {1,0,0,1},
    {1,0,0,1},
    {0,1,1,0}
};

int countBits(int mask){
    int c=0;
    while(mask){
        c += mask & 1;
        mask >>=1;
    }
    return c;
}

int isTotalDominating(int mask){
    for(int v=0; v<N; v++){
        int covered=0;
        for(int u=0; u<N; u++){
            if((mask&(1<<u)) && graph[u][v]){
                covered=1;
                break;
            }
        }
        if(!covered) return 0;
    }
    return 1;
}

void printSet(int mask){
    printf("{");
    int first=1;
    for(int i=0;i<N;i++){
        if(mask&(1<<i)){
            if(!first) printf(",");
            printf("%d",i+1);
            first=0;
        }
    }
    printf("}");
}

int main(){
    int min=N+1;

    printf("Total Dominating Sets:\n");

    for(int mask=1; mask<(1<<N); mask++){
        if(isTotalDominating(mask)){
            printSet(mask);
            printf("\n");

            int size=countBits(mask);
            if(size<min)
                min=size;
        }
    }

    printf("\nTotal Domination Number = %d\n",min);

    return 0;
}

```



```

// File: Lab12_Graph_Matrices.c

// Objective: To construct different matrices associated with graphs.
// Problem: Write a program to construct: Adjacency Matrix, Degree Matrix, and Laplacian Matrix
// Input: N = 5, M = 4, Edges = {(1,2), (2,3), (4,5), (1,5)}
// Output
// Adjacency, Degree, and Laplacian matrices.
#include <stdio.h>

#define N 5

int main() {
    int adj[N][N] = {0};
    int degree[N][N] = {0};
    int laplacian[N][N];

    int edges[4][2] = {
        {1,2}, {2,3}, {4,5}, {1,5}
    };

    for(int i=0; i<N-1; i++) {
        int u = edges[i][0] - 1;
        int v = edges[i][1] - 1;
        adj[u][v] = 1;
        adj[v][u] = 1;
    }

    for(int i=0; i<N; i++) {
        int deg = 0;
        for(int j=0; j<N; j++)
            deg += adj[i][j];
        degree[i][i] = deg;
    }

    for(int i=0; i<N; i++)
        for(int j=0; j<N; j++)
            laplacian[i][j] = degree[i][j] - adj[i][j];

    printf("Adjacency Matrix:\n");
    for(int i=0; i<N; i++) {
        for(int j=0; j<N; j++)
            printf("%d ", adj[i][j]);
        printf("\n");
    }

    printf("\nDegree Matrix:\n");
    for(int i=0; i<N; i++) {
        for(int j=0; j<N; j++)
            printf("%d ", degree[i][j]);
        printf("\n");
    }

    printf("\nLaplacian Matrix:\n");
    for(int i=0; i<N; i++) {
        for(int j=0; j<N; j++)
            printf("%d ", laplacian[i][j]);
        printf("\n");
    }

    return 0;
}

```



```

// File: Lab15_Shortest_Paths.c

// Objective: To find shortest paths between all pairs of vertices.
// Problem: Write a program to implement Floyd-Warshall Algorithm.
// Input:
// Weighted graph represented using adjacency matrix.
// Output:
// Shortest path matrix between all pairs of vertices.
#include <stdio.h>

#define N 4
#define INF 99999

int main()
{
    int graph[N][N] = {
        {0, 5, INF, 10},
        {INF, 0, 3, INF},
        {INF, INF, 0, 1},
        {INF, INF, INF, 0}
    };

    int dist[N][N];

    for (int i = 0; i < N; i++)
        for (int j = 0; j < N; j++)
            dist[i][j] = graph[i][j];

    for (int k = 0; k < N; k++)
        for (int i = 0; i < N; i++)
            for (int j = 0; j < N; j++)
                if (dist[i][k] != INF && dist[k][j] != INF &&
                    dist[i][k] + dist[k][j] < dist[i][j])
                    dist[i][j] = dist[i][k] + dist[k][j];

    printf("Shortest Path Matrix:\n\n");

    for (int i = 0; i < N; i++)
    {
        for (int j = 0; j < N; j++)
        {
            if (dist[i][j] == INF)
                printf("INF ");
            else
                printf("%3d ", dist[i][j]);
        }
        printf("\n");
    }

    return 0;
}

```